

Unified Tool Provider Contract

The RAGenius Execution Subsystem invokes tools through a unified interface called the **Tool Engine**. Tools encapsulate the low-level work required to fulfil a step in a skill workflow—such as reading a file, calling an external API, running a script, or retrieving data from RAG. This document specifies the contract for defining and executing tools so that developers can add new tools without modifying the core engine.

Provider Types

A tool belongs to exactly one **provider type**, which determines how it is invoked:

Provider Type	Description	Typical Examples
<code>local</code>	Runs locally within the execution environment. Useful for file operations, simple computations, or sandboxed Python execution.	<code>file.read</code> , <code>python.execute</code>
<code>api</code>	Invokes an external REST or SDK API over the network. Often requires credentials or API keys.	<code>sendgrid.send_email</code> , <code>stripe.create_charge</code>
<code>mcp</code>	Discovered from a Multi-Call Provider (MCP). MCPs expose their own set of tools. The engine must map these tools into the internal schema before use.	<code>mcp.notion.create_page</code> , <code>mcp.asana.create_task</code>
<code>rag_adapter</code>	Wrapper around the RAG subsystem. Only supports read-only retrieval; ingestion or mutation is forbidden.	<code>rag_retrieval_tool</code>

Tool Definition

Each tool must provide a manifest with these properties:

Field	Type	Description
<code>id</code>	string (unique)	Unique identifier for the tool (e.g. <code>mock_video_generation_tool</code>).
<code>name</code>	string	Human-friendly name.
<code>providerType</code>	string	One of <code>local</code> , <code>api</code> , <code>mcp</code> , or <code>rag_adapter</code> .

Field	Type	Description
<code>inputSchema</code>	JSON Schema object	Schema describing the expected input. The tool engine must validate inputs against this before execution.
<code>outputSchema</code>	JSON Schema object	Schema describing the output. Used for result normalization.
<code>permissionScopes</code>	array of strings	Permission scopes required to call this tool (e.g. <code>filesystem.write</code> , <code>external_api.write</code>).
<code>timeoutMs</code>	integer (optional)	Maximum time to wait for the tool to complete. Default should be reasonable (e.g. 30s).
<code>sideEffecting</code>	boolean	Whether calling this tool can create external side effects (file writes, network mutations, etc.). Side-effecting tools must be restricted or require confirmation.
<code>metadata</code>	object (optional)	Free-form metadata about the tool (e.g. provider endpoint, description, version).

Example Tool Manifest (JSON)

```
{
  "id": "mock_video_generation_tool",
  "name": "Mock Video Generation Tool",
  "providerType": "api",
  "inputSchema": {
    "type": "object",
    "required": ["prompt", "duration"],
    "properties": {
      "prompt": { "type": "string" },
      "duration": { "type": "number", "minimum": 1, "maximum": 300 },
      "context": { "type": "string" }
    }
  },
  "outputSchema": {
    "type": "object",
    "required": ["title", "summary", "file_id"],
    "properties": {
      "title": { "type": "string" },
      "summary": { "type": "string" },
      "file_id": { "type": "string" }
    }
  },
  "permissionScopes": ["external_api.write"],
  "timeoutMs": 30000,
}
```

```

"sideEffecting": true,
"metadata": {
  "endpoint": "https://mock-video.example.com/generate",
  "provider": "MockVideoAPI"
}
}

```

Execution Contract

1. **Input validation:** The tool engine must validate the input against `inputSchema` before invoking the tool. Invalid inputs result in a `validation` error and no call is made.
2. **Permission check:** Before each call, the permission engine must verify that the execution context has all `permissionScopes` declared by the tool. If a scope is `restricted` or `require_confirmation`, the engine must pause or block the call accordingly.
3. **Timeout enforcement:** If the tool does not complete within `timeoutMs`, the engine should terminate the call and classify the error as `timeout`. Retries may be attempted based on configured retry policy.
4. **Provider invocation:**
 5. For `local` tools, call the local adapter with the validated input.
 6. For `api` tools, call the configured endpoint or SDK. The engine must inject credentials from environment variables or secrets manager and must not log raw tokens.
 7. For `mcp` tools, call the MCP client with the mapped tool metadata. The MCP layer should handle authentication, discovery and error mapping.
 8. For `rag_adapter` tools, call the RAG subsystem in read-only mode with the provided query. No ingestion or mutation operations are allowed.
 9. **Result normalization:** The raw output should be validated against `outputSchema` and then mapped into the step's expected output fields. The engine should return only the data needed by the workflow and omit any sensitive metadata.
 10. **Error handling:** Errors must be classified as one of `validation`, `permission`, `tool`, `timeout`, or `external_api`. Each error should include a code, message, details, recoverability flag, and suggested action.
 11. **Logging:** Log the tool call with execution ID, tool ID, provider type, input summary, output summary, error class, duration, and whether redaction was applied. Do not log secrets, full payloads or raw tokens.

Provider-Specific Notes

Local Providers

- Should run in a sandboxed environment with resource limits (CPU, memory, disk).
- File system tools must be constrained to allowed directories.
- Code execution tools (e.g. Python) should be limited to restricted modules and not allow arbitrary network access unless explicitly allowed.

API Providers

- Credentials (API keys, tokens) must be sourced from environment variables or secret management. They must never be hard-coded or logged.
- The tool metadata should include endpoint URL and any required headers.
- Consider adding retry policies and exponential backoff to handle transient failures.

MCP Providers

- The MCP integration layer is responsible for discovering tools and mapping them into the internal schema. Each discovered tool must be assigned a unique `id`, `providerType` of `mcp`, and appropriate `permissionScopes` based on the provider's capabilities.
- Unknown MCP tools must not be executed until explicitly allowed and registered.
- Use caching to reduce discovery calls, and handle provider downtime gracefully.

RAG Adapter Providers

- Only read-only retrieval operations are allowed. Ingestion, mutation, deletion, or index modification operations must never be exposed through a `rag_adapter` tool.
- Inputs typically include a `query` string and optional `topK` parameter.
- Outputs should return items with fields like `title`, `content`, and optional `metadata`.
- The adapter should support simple filtering or ranking parameters but should never influence the planning logic.

Tool Registration

Tools should be registered via a registry service or CLI with their manifests. Registration should validate the manifest, assign the tool to the correct provider type, and ensure there are no ID collisions. Once registered, tools can be discovered by skills via the `required_tools` list.

Deprecation and Removal

- Mark tools as `enabled: false` to prevent new executions while still allowing historical logs to be reviewed.
 - Before removing a tool completely, migrate or disable any skills that depend on it.
 - Document the reason for deprecation and possible alternatives.
-